

Compiler Symbol Table and Parser Notes

Files and Relationship

``23101272.y`` → `symbol_table.h` → `scope_table.h` → `symbol_info.h``

- ``symbol_info.h``: stores information about one symbol.
- ``scope_table.h``: stores symbols belonging to one scope.
- ``symbol_table.h``: manages all scopes.
- ``23101272.y``: parses the program and performs symbol-table operations.

1. symbol_info.h

Purpose

The ``symbol_info`` class represents one symbol, such as a variable, array, function, token, or grammar item.

Main Attributes

- ``name``: actual symbol name, such as ``x``, ``marks``, or ``add``.
- ``type``: general token type, such as ``ID``, ``INT``, or ``FLOAT``.
- ``symbol_category``: identifies whether the symbol is a ``Variable``, ``Array``, or ``Function Definition``.
- ``data_type``: stores a variable/array type or function return type, such as ``int``, ``float``, ``char``, or ``void``.
- ``parameters``: ``vector`` containing function parameter type and name.
- ``array_size``: stores an array length.

Example:

```
int x;
```

```
float marks[50];
```

```
int add(int a, int b);
```

Stored information:

- ``x`` → Variable, int
- ``marks`` → Array, float, size 50
- ``add`` → Function Definition, return type int, parameters ``(int,a)`` and ``(int,b)``

Important Methods

- ``get_name()`` and ``get_type()`` return basic information.

- ``set_name()`` and ``set_type()`` modify basic information.
- ``get/set_symbol_category()`` manage the category.
- ``get/set_data_type()`` manage the data type.
- ``get/set_parameters()`` manage the parameter list.
- ``add_parameter(type,name)`` adds one parameter.
- ``get_parameter_count()`` returns the number of parameters.
- ``get/set_array_size()`` manage the array size.

Constructor

``symbol_info(name,type)`` initializes the name and type. The category and data type are empty, and ``array_size`` is ``-1``.

Key Idea

``symbol_info`` is the information record for one symbol.

2. scope_table.h

Purpose

A ``scope_table`` represents one scope and stores all symbols declared in that scope.

Example:

```
int x;

int main() {
    int y;
    {
        int z;
    }
}
```

Hierarchy:

Global scope → Function scope → Inner block scope

Main Attributes

- ``bucket_count``: number of hash-table buckets.
- ``unique_id``: identifier of the scope.
- ``parent_scope``: pointer to the outer scope.

- ``table``: ``vector>``; the vector stores buckets and each list handles collisions.

Hash Function

The file uses a polynomial rolling hash:

```
`hash_value = hash_value * 31 + character`
```

The final bucket is:

```
`hash_value % bucket_count`
```

This converts a symbol name into a valid bucket index.

Important Methods

``lookup_in_scope(symbol)``

Searches only the current scope. It calculates the bucket, compares names, returns the stored symbol if found, and returns ``NULL`` otherwise.

``insert_in_scope(symbol)``

Checks for ``NULL``, prevents duplicate names in the same scope, calculates the bucket, and inserts the symbol.

``delete_from_scope(symbol)``

Finds the symbol in the current scope, deletes the object, and removes its pointer from the list.

``print_scope_table(outlog)``

Prints the scope ID, non-empty buckets, and symbol details. Variables show their type; arrays show type and size; functions show return type and parameters.

Destructor

Deletes all symbol objects and clears the lists and vector.

Key Idea

``scope_table`` is one hash table for one scope.

3. symbol_table.h

Purpose

``symbol_table`` manages the complete hierarchy of scope tables.

Main Attributes

- ``current_scope``: points to the active scope.
- ``bucket_count``: used when creating new scopes.
- ``current_scope_id``: generates scope IDs.

Important Methods

``enter_scope()``

Creates a new scope. The old current scope becomes the parent.

``exit_scope()``

Moves ``current_scope`` to its parent and deletes the old current scope.

``insert(symbol)``

Inserts into the current scope by calling ``current_scope->insert_in_scope(symbol)``.

``remove(symbol)``

Removes from the current scope by calling ``delete_from_scope(symbol)``.

``lookup(symbol)``

Searches:

Current scope → Parent scope → Grandparent scope → ``NULL``

This allows an inner block to use symbols declared in outer blocks.

Printing

- ``print_current_scope()`` prints only the active scope.
- ``print_all_scopes()`` prints from the current scope to the outermost scope.

Destructor

Repeatedly exits scopes until all scopes are removed.

Key Idea

``symbol_table`` is the manager of all scope tables.

4. 23101272.y

Purpose

This is the Bison/Yacc parser file. It:

- defines grammar rules,
- receives tokens from ``23101272.l``,
- performs semantic actions,
- creates and removes scopes,
- inserts variables, arrays, and functions,
- writes parsing information to the log.

Compiler Flow

Source code → Flex/Lex (``23101272.l``) → Tokens → Bison/Yacc (``23101272.y``) → Symbol-table operations → Log output

Important Declarations

- ``#include "symbol_table.h"`` gives the parser access to all symbol-table classes.
- ``#define YYSTYPE symbol_info*`` means grammar values use ``symbol_info*``.
- ``$1``, ``$2``, and similar names refer to values on the right side of a grammar rule.
- ``$$`` stores the value produced by the current rule.
- ``int lines = 1`` stores the current line number.
- ``ofstream outlog`` writes logs.
- ``symbol_table *table`` is the main symbol table.
- ``current_data_type`` temporarily stores the currently declared type.
- ``current_parameters`` temporarily stores function parameters.
- ``function_scope_pending`` indicates that parameters should be inserted when a function scope is created.

Important Grammar Rules

``start``

Represents the complete program and prints the symbol table after parsing.

``program``

A program contains one or more units:

``program : program unit | unit``

``unit``

A unit can be a variable declaration or function definition.

``func_definition``

For:

```
int add(int a, int b) {  
  return a+b;  
}
```

the parser stores:

- name: ``add``
- category: ``Function Definition``
- return type: ``int``
- parameters: ``(int,a)`, `(int,b)``

``param_list``

Stores parameter type and name. An unnamed parameter is stored with an empty name.

``compound_statement``

When ``{`` is read:

1. ``enter_scope()`` creates a scope.
2. Function parameters are inserted when needed.

When ``}`` is read:

1. Scopes are printed.
2. ``exit_scope()`` removes the current scope.

``type_specifier``

Recognizes types such as ``int``, ``float``, ``void``, and ``char``, then updates ``current_data_type``.

``declaration_list``

Handles:

```
int x;
```

```
int x,y,z;
```

```
int numbers[10];
```

Variables receive category ``Variable``. Arrays receive category ``Array``, a data type, and an array size.

Statements

The grammar supports declarations, expressions, compound statements, ``for``, ``if``, ``if-else``, ``while``, ``println``, and ``return``.

Dangling Else

``%nonassoc LOWER_THAN_ELSE`` and ``%nonassoc ELSE`` make an ``else`` associate with the nearest unmatched ``if``.

Expression Precedence

``expression`` $\rightarrow$ ``logic_expression`` $\rightarrow$ ``rel_expression`` $\rightarrow$ ``simple_expression`` $\rightarrow$ ``term`` $\rightarrow$ ``unary_expression`` $\rightarrow$ ``factor``

Therefore, ``a + b * c`` is grouped as ``a + (b * c)``.

Quick Revision

| File | Responsibility |

|---|---|

| ``symbol_info.h`` | Stores one symbol |

| ``scope_table.h`` | Stores symbols in one scope using a hash table |

| ``symbol_table.h`` | Manages all scopes and searches parent scopes |

| ``23101272.y`` | Parses the program and controls symbol-table operations |

Viva Questions

1. What is ``symbol_info``? A class storing one symbol's name, category, type, parameters, and array size.
2. What is a scope table? A hash table for symbols declared in one scope.
3. Why is ``parent_scope`` needed? It links inner scopes to outer scopes for lookup.
4. Difference between ``lookup_in_scope()`` and ``lookup()``? The first searches one scope; the second searches the current scope and all parents.
5. Why use hashing? To map names to buckets for efficient storage and searching.

6. What does ``enter_scope()`` do? Creates and activates a new scope.
7. What does ``exit_scope()`` do? Moves to the parent and deletes the old scope.
8. What is ``YYSTYPE`` here? ``symbol_info``.
9. What is ``current_data_type`` used for? It stores the type currently being declared.
10. What is ``function_scope_pending`` used for? It signals that function parameters must be inserted into a new function scope.

Memory Trick

One symbol → One scope → All scopes → Parser control

``symbol_info`` → `scope_table` → `symbol_table` → 23101272.y`